V8-microRISC Software Development Tools

September 30, 1998

Disclaimer

VAutomation Reserves the right to make changes at any time, without notice, to improve design or performance and supply the best product possible. VAutomation Inc. makes no warrant for the use of its products and assumes no responsibility for any errors which may appear in this document nor does it make a commitment to update the information contained herein. VAutomation products are not authorized for use in life support devices or systems unless a specific written agreement pertaining to such use is executed between the manufacturer and an officer of VAutomation. Nothing contained herein shall be construed as a recommendation to use any product in violation of existing patents, copyrights or other rights of third parties. No license is granted by implication or otherwise under any patent, patent rights or other rights, of VAutomation Inc. All trademarks are trademarks of their respective companies.

VAutomation Inc.
20 Trafalgar Square
Suite 443
Nashua NH 03063
(603) 882-2282
FAX: 882-1587
http://www.VAutomation.com

Table of Contents

Introduction

The V8-microRISC 8-bit microprocessor is fully supported with a set of software development tools. These tools include an Assembler, Instruction Set Simulator, a JTAG hardware debugger and an ANSII standard C compiler. Each of these tools is described in detail in the following chapters.

This manual assumes that the reader is familiar with the architecture and instruction set of the V8. For more details on the V8, refer to the following documentation.

Reference Documents

V8-microRISC Reference Manual

The V8 Reference manual provides detail on the architecture, instruction set and programmers model. V8 peripherals are also described in the reference manual.

Intellicore Prototype System

The Intellicore Prototype System (IPS) is an FPGA based evaluation and prototyping system for V8 based systems. The IPS is currently used as the JTAG master for debugging a target application which can either be another IPS or actual silicon which includes the JTAG debugging logic.

HiTech Software V8-microRISC C compiler

The V8-microRISC C compiler is available from HiTech software. Details on the C compiler can be obtained by contacting HiTech at HTTP://www.htsoft.com or by calling them in Australia at +61-7-3354-2411

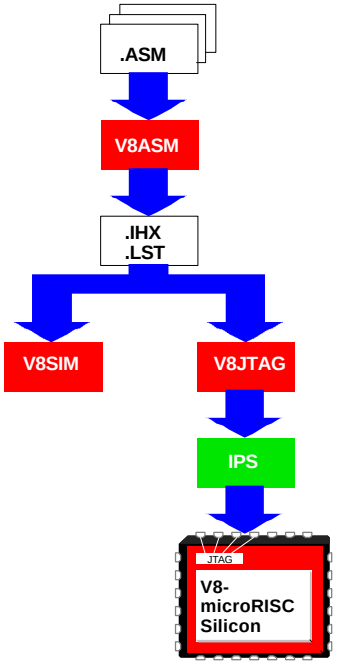
Tool Flow

Every design team will want to develop their own software development tool flow, the diagrams below explain two basic tool flows that we use here at VAutomation to develop applications.

Assembly Source Code

Assembly language is ideal for small applications which are expecting less than 4 kilobytes of code to be required. The assembly tools come with the V8-microRISC CPU for free and are adequate for small applications. Assembly gives the programmer maximum control over every step of the program and generally results in the smallest, fastest and most efficient code. However, assembly is difficult to read and maintain and is not recommended for large applications.

The diagram to the right shows a typical tool flow using assembly as the source code. A single top level assembly source code file with a .ASM extension is the primary starting point. This file in turn can contain .include assembler directives to bring in other assembly source files. Note that there is no linker for the assembler. It is assumed that the project will be fairly small when using assembly so a linker is not required.



The assembly source files are assembled using the V8ASM program. V8ASM generates a Intel-HEX format object file (.IHX) and an assembly listing file (.LST). During the initial phases of debugging, the instruction set simulator is typically used. The .IHX and .LST files are input into V8SIM for complete source code level debugging. Once the code is initially debugged, it can then be loaded into the V8JTAG debugger which in turn downloads the code to an IPS based JTAG master which in turn downloads the code directly into the target silicon. The target silicon could be another IPS for early software development before the final silicon is available.

C Source Code

C should be used as the software source for any project that expects to develop more than 4 Kilobytes of code. Projects of this size will benefit significantly from the efficiences of coding in a high level language such as C.

V8ASM V8 Assembler

One of the primary features of the V8 is its' ability to have new opcodes added to the instruction set. A highly configurable assembler is provided in C source code form to allow users to take maximal advantage of this feature of the V8. Applications which make major changes to the V8 instruction set may want to modify the V8ASM C source code. However, applications which are simply adding new definitions to the two user-definable V8 opcodes can simply edit the V8ASM.SYN syntax file. The V8ASM.SYN file describes the syntax and bit fields of the V8 opcodes. Any or all of the standard opcodes can be modified by simply changing the fields in the V8ASM.SYN file.

The V8ASM.C file is written in ANSI standard C and can be compiled with any ANSI compatible C compiler such as the GNU GCC, Microsoft or Borland C compilers. Executables for Sun Unix and DOS are provided. The assembler is command line only, there is no GUI.

For examples on coding syntax in the V8ASM assembly language, see the V8_DEMO.ASM file which has examples of all the assembler directives and opcodes.

V8ASM requirements:

Inputs:

v8asm.syn	Syntax file which must be in the same directory as the executable V8ASM file.
Progfile	The assembler source file and all of the include files must be in the same directory.

Outputs:

Progfile.lst	Assembler listing file containing the source file, locations, and object code.
Progfile.ihx	The object file in Intel Hex format

Usage: **`v8asm [-dSYM=val] [-l] [-pnn][-s] [-tnn] progfile`**

All of the options in [] are optional.

-d defines a symbol. Multiple -d options can be used on the command line. The symbol specified between the -d and the = is assigned the value VAL. This option is most commonly used for conditionally compiling.

-l turns off “lint” detection in the assembly source. By default the assembler will issue warnings about various constructs that may not be correct such as doing a CMP R0 which will compare R0 with R0 which naturally will always be true.

-p sets the number of lines on each page in the listing file. The default is 22222 lines.

-s will dump the symbol usage at the bottom of the listing file. This is handy when looking for symbols that are defined but not used. Their usage count will be 1.

-t sets the number of spaces for each tab. Defaults to 0 so no tabs will be in the listing file. Setting to 4 will make the listing file much smaller.

progfile is the file name to assemble, if the name does not have a suffix, it is automatically suffixed '.asm'.

Examples:

v8asm
Will display the program revision and usage syntax message.

v8asm myprog
Will assemble myprog.asm and create myprog.ihx and myprog.lst

v8asm -p60 myprog.a
Will assemble file myprog.a and produce myprog.lst with 60 lines per page.

v8asm -dUSB_ENABLE=1 myprog.a
Will assemble file myprog.a and produce myprog.lst. The symbol USB_ENABLE is assigned the value “1" prior to assembly.

The V8ASM assembler produces two files upon successful assembly of the source files, the .LST listing file and the .IHX Intel Hex format file. Each line of the listing file contains a 4 character field with the current hex address location, up to 8 hex characters of the data at that location, a 5 digit field with the source code line number followed by the line from the source code.

Sample Listing file:

```
                VAutomation V8 RISC Assembler                  Page 1
                  Version 1.0, Oct  7 1997

 Loc     Code     Stmt  Source Code
                      |....+....1....+....2....+....3....+....4....+..
                             .org 0x410
 0410    BF 3908   185       JSR UART_INIT ; init the UART for 19.2Kb
 0413    BF 890C   186       JSR AUDIO_INIT    ; init the AD1848 chip
 0416    BF 4E06   187       JSR PP_INIT ; Init the P1284 port
 0419    BF EE0D   188       JSR USB_INIT ; init the USB interface
```

Each line of the Intel Hex file consists of the following fields in the columns specified:

1:
2ll ; Two digit number of bytes of data
4aaaa ; Four digit starting address
8tt ; Two Digit record type (00=code or data, 01=end of file, 02=upper segment address)
10data ; zero to 32 digits of data
n cc; two digit checksum (twos complement of (0xff logcially ANDed with the sum of all data bytes))

Example .IHX file:

```
:020000020000FC
:10040000F40DF70DF70D020EF70DF70DF70DF70DC3
:01107000314E
:00000001FF
```

V8ASM.SYN V8 Opcode Syntax Definition File

The V8ASM.SYN file provides the definitions of the opcodes for the V8. See the V8ASM.SYN for an example of how the standard opcodes are defined. The V8ASM.SYN file also shows how opcodes can be aliased to more meaningful names IE: BR0 0,nnn is aliased to BRNE nnn which makes it more obvious that this is a Branch if Not Equal. The syntax of V8ASM.SYN file is:

1. opcode in column 1 and 2 in hex (00 to ff)
2. a comma delimiter
3. mnemonic of instruction (max 4 chars)
4. a comma delimiter
5. a 2 byte operand format designator

operand formats are:

0 = no operand
1 = register operand r0-r7, R0-R7 to be placed in the first 3 bits of opcode
2 = three bit number 0-7 to be placed in first 3 bits of opcode
3 = 8 bit operand to be placed as 2nd byte of object code
formats are:
127 to -128
0x00 to 0xff
h'00 to h'ff
b'00000000 to b'11111111
o'000 to o'776
'c' a single character
"c" a single byte string
Label as offset from current position
4 = Same as 3, except Labels are treated as a value
5 = 16 bit operand to be placed as 2nd and 3rd byte of object code
formats are:
0 - 65535
0x0000 to 0xffff
h'0000 to h'ffff
b'0000000000000000 to b'111111111111111
o'000000 to o'777774
'c' a single byte character
"cc" two byte string

Example:

98,br1,23; Branch if PSR bit=1 opcode=0x98 + a 3 bit bit field (the 2 after the comma) and requires another byte as a relative offset (the 3 after the 2).

98,breq,30; This is another way of hard coding the above opcode where the bit field is zero (IE: we are testing the Z bit of the PSR. The opcode is still a 98, we don't need the 3 bit field since we have hard coded it, and we only need a one byte relative branch offset as indicated by the 3 after the comma.

V8ASM Syntax

The V8ASM assembler source files contain statements with zero to four fields in the following form:

[LABEL] [OPCODE MNEMONIC] [EXPRESSION] [COMMENTS]

All of the fields are optional. All lines in the file must be less than 250 characters long.

Label

Labels are symbolic names that are used to equate the starting address of the code generated at this line of the assembly source file. Labels must be less than 250 characters in length and must be terminated with a ":" (colon) character. The label may contain any valid ASCII printable character.

Example:

THIS_IS_A_LABEL:; Valid label
THIS BAD label:; invalid due to spaces in the label name.
THIS_IS_BAD :; Invalid due to space between the label and the :

Opcode Mnemonic

The Opcode Mnemonics are defined in the V8ASM.SYN file. The assembler uses the mnemonic to identify the corresponding bit pattern for the opcode and to determine any additional fields for the opcode.

Expression

Expression requirements depend on the opcode. The following operators are supported:

'+'addition
'-' subtraction
'*'multiplication
'/'division
'>>'Shift right
'<<'shift left
'='equality

All expressions evaluate to 16 bit integer values. Standard notation for integers is supported:

Hexadecimal0xnnn | hnnn
Decimalnnn | -nnn
Octalo'nnn
Binaryb'nnn

Example:

```
START:lda    r0, BASE_ADDRESS+REGISTER_22    ; load R0 from memory
             ldir5, IO_REG_BASE>>8                    ; load R4/R5
                                  pair with the index to an IO reg
             ldir4, IO_REG_BASE
             stxr4                                    ; Store R0 to the IO reg
                                  pointed to by R4/R5
```

Comments

Comments begin with the semi-colon ';' character and continue to the end of the line. Any text may be written after the semi-colon character to increase the readability of the assembly program.

V8ASM Assembler Directives

All assembler directives start with the period '.' character. Note that the date/time directives start with two periods.

.include "filename"

The include directive causes the assembler to insert the assembly source code file into the current location of the current assembly file. This allows large programs to be broken into multiple small files which are easier to manage.

Example:.include "submodule.asm" ; will include all of submodule.asm here

.org <expression>

The current assembler address to value of the expression. Object code overlays are not allowed. Attempts to produce object code over an area in the program where object code already exists will produce an error.

Example:

```
.org 0x100
.string "This is a string at 0x100 in memory"
.org 0xfe
.string "This is a string at 0xfe in memory"
```

will produce an error since we are trying to place 2 different values in location 0x100 in memory.

.equ <identifier> <expression>

The identifier is equated to the value in the expression. Attempts to re-equate a value to a new value produces an error UNLESS the new equated value exactly matches the previously equated value. For

Example:

```
.equ aaa bbb
.equ aaa ccc   ; This will produce an ERROR

.equ aaa bbb
.equ aaa bbb   ; This will NOT produce an ERROR
```

Duplicate equates are allowed so assembler code containing equate values can be included multiple times without causing problems. Attempts to equate a value which is defined as a branch label produces an error.

Example:

```
.org 0x100
LABEL_AT_100:
.equ LABEL_AT_100 LABEL_AT_200   ; This will produce an error!
.equ ERROR_LABEL LABEL_AT_100    ; This is perfectly acceptable
```

.byte <expression>, .word <expression>, .rword <expression>

The .byte directive reserves one byte of memory and assigns it the value in the expression. The .word directive reserves two bytes with the most significant byte of expression at the first memory location and least significant byte at the second. The .rword directive is the same as the .word except the bytes are swapped.

Example:

```
AddrData        Line     Source Code
          100   .org 0x1234
12345678        101      THIS_LABEL: .byte 0x56, h78
12363412        102      .word THIS_LABEL
12381236        103      .rword THIS_LABEL+2
```

.string "xxx"

The .string directive inserts the ASCII character string xxx in the object code. The maximum length string is currently 254 characters. Any valid ASCII character can be placed in the string. The escape character backslash '\' is used to insert various non-printable ASCII characters into the string. If an escape character is found in the string, the escape character is removed from the string and the character immediately following the escape character is treated as shown in the table below. Any other character following the escape character is taken literally. Unlike the .byte and .word directives, no other formats (such as .string 0x0a0b0c) are accepted with the .string directive.:

Escape char	Hex Value	Description
\a	07	audio alert
\b	08	Backspace
\f	0C	form feed
\n	0A	Newline
\r	0D	carriage return
\0	00	NULL
\t	09	horizontal tab
\v	0B	vertical tab
\\	5C	single backslash character
\'	27	single quote
\"	22	double quote character

.unicode "xxx"

The .unicode directive functions identically to the .string directive except that it will store the string in UNICODE format. UNICODE for ASCII text simply uses two bytes for each character with the upper byte being zero and the lower byte being the ASCII character. Strings in Universal Serial Bus devices must be in UNICODE format.

..datetime

Date and time strings are supported with the following syntax. These directives must be coded on a line by themselves. The ASCII characters that make up the date or time strings are inserted into the object code. The assembler makes a system call to compute the current date and time. These strings are useful for embedding the build date of an assembly program.

..dateyyyymmdd
..timehhmmss
..datetimeyyyymmddhhmmss
..yearyyyy
..monthmm
..daydd
..hourhh
..minutemm
..secondss

.list, .nolist

The .LIST directive causes the assembled source listing to be sent to the .LST file output file. By default, the source is sent to the .LST file. The .NOLIST causes the assembled source listing to be suppressed. Macros and large tables of data are often not included in the listing file to reduce it's size.

.ram [addr] , .rom [addr]

The .RAM directive tells the assembler to place all following code or variables into the RAM space. The .ROM directive tells the assembler to place all following code into ROM space. These two directives allow you to mix RAM variables into your ROM source code without having to constantly reORG between the RAM and ROM spaces. Each directive takes an optional address parameter which sets the current RAM or ROM address.

```
EX:.RAM 0x8000          ; set the RAM space to be up at 0x8000
                                        .ROM 0x0000; set ROM to be at 0
        LDA r0, A_ROM_CONSTANT
        STA r0, A_RAM_VARIABLE
        ...
                ..ram
        A_RAM_VARIABLE: .byte 0 ; at 0x8000
        .rom
        A_ROM_VARIABLE: .byte 0x99; a byte a few bytes up from 0x0000
        ...
```

.if, .ifn, .ifdef, .ifndef, .else .endif

The assembler supports conditional compilation with the use of these directives.

```
.if<variable>      ; code is assembled if VARIABLE evaluates to true (nonzero)
.ifn<variable>     ; code is assembled if VARIABLE evaluates to false (zero)
.ifdef<variable> ; code is assembled if VARIABLE is defined
.ifndef   <variable>        ; code is assembled if VARIABLE is NOT defined
```

Structure of the if-else-endif is similar to that used in C. Variables can be defined on the assembler command line using the -d option. The following examples demonstrate the conditional compilation syntax.

Example 1:

```
.if CODE_ENABLED; if CODE_ENABLED is not equal to zero, then the string
.string “The code is enabled”;     is included in the assembly listing and hex file.
.endif;                                    Otherwise, it is not.
```

Example 2:

```
.ifndef CODE_ENABLED: if CODE_ENABLED has NOT been defined, then the string
.string “The code is enabled”;  is included in the assembly listing and hex file.
.endif; Otherwise, it is not.
```

Example 3:

```
.ifdef VERSION_A
        .string “This string is included on VERSION A”
.else
 .if VERSION_B; IFs can be nested but they must be on separate lines
        .string “This string is included on VERSION B”
 .else
        .string “This string is included on any other VERSION”
 .endif
.endif
```

Macros

The V8ASM assembler supports macros with the following syntax:

```
.macro <macro_name>
```

```
        <macro code>
                %1 through %9 can be used as arguments to the macro
                ~ becomes the macro-name_instance
        .endmacro
```

Example:

```
                                        .macro jeq; macro for long branches
                brne ~skip
                jmp %1
        ~skip:
        .endmacro
```

To invoke the macro in the assembly file:

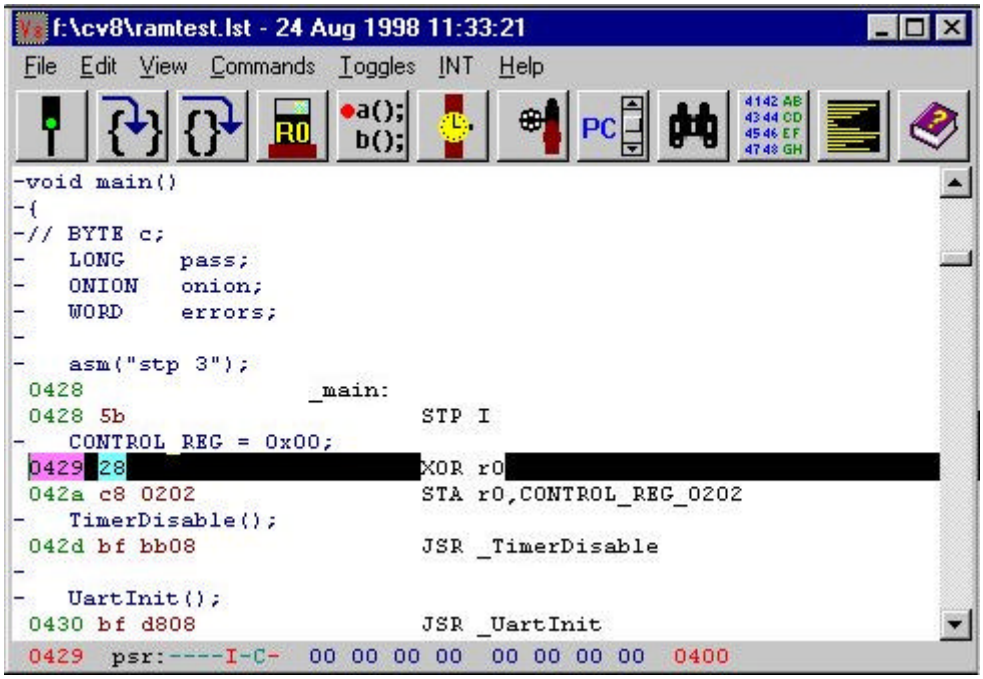
```
        jeq foobar
```

is expanded by the assembler to:

```
                brne jeq_1_skip
                jmp foobar
        jeq_1_skip:
```

The V8SIM simulator

The V8-uRISC Software Simulator is a Win32 application that loads and executes an Intel hex file. V8SIM provides a high speed debugging interface for quickly debugging V8 software. V8SIM allows software debug to begin before the V8 has been realized in silicon form. This allows the software and hardware development teams to operate in parallel to bring your product to market faster.



V8SIM is entered by double clicking on the V8SIM.EXE file.

Use the FILE menu to Open a .LST file. Any .LST file from the V8ASM assembler can be loaded into the Simulator.

V8SIM opens two windows: a main window as shown above and a serial I/O (UART) window. The main window displays the .LST file and highlights the line currently being executed by the V8-uRISC. The UART window is a transcript window that displays serial communications to/from a UART. Commands can be sent to the UART by simply clicking on this window and then typing commands.

At the bottom of V8SIM's main window is a status bar that displays the current status of various registers in the V8-uRISC. From left to right, the fields are:

Current Program Counter (PC)
Stack Pointer
Program Status Register
- Bits that are zero are displayed as a '-'
- Bits that are one are displayed with their mnemonic character (7654INCZ)
- Bits that changed since the last opcode are displayed in red

Registers R0-R7

The buttons do the following (from left to right):

Start/Stop the processor.
Execute the next instruction/statement.
Execute the next instruction/statement but treat subroutines as one instruction.
Show/Hide the processor state window.
Show/Hide the breakpoint window.
Show/Hide the watchpoint window.
Reload the .hex and .lst files.
Scroll the source window until the program counter is displayed.
Find text in the source window.
Show/Hide the ram dump window.
Show/Hide the profile window.
Display help.

Menus

File Menu

Open - lets you browse for a .lst file to open.
Reload - reloads the current file. Useful if you've reassembled it.
Font - lets you specify the font size and style. Fixed fonts usually look best.
Color - lets you change the window colors.
Exit - terminates V8SIM.

Edit Menu

Edit source - opens the file which contains the currently selected line and places the caret there. CodeWright is the default editor. The editor may be specified in the registry: HKEY_LOCAL_MACHINE\ Software \VAutomation \Editor. This command assumes the editor understands the -G### switch.
Find - prompts you for a string to search for.
Find next - searches for the next occurance of the most recently specified string.
Clear comm window – removes all text from the comm window.

View Menu

Breakpoints – Show/Hide breakpoint window.
Comm port activity – Show/Hide comm port activity.
Dump ram - Shows/Hides the ram dump window.
Histogram - Shows/Hides the histogram window.
Processor State – Show/Hide processor state window.
Watchpoints – Show/Hide watchpoint window.

Commands Menu

Step 1 - executes one instruction/statement.
Step 20 - executes twenty instructions/statements.
Step over JSR - executes one instruction/statement, subroutines count as one.
Go - instructions are executed begining with the current one.
Stop - execution is halted.
Scroll to PC – Scroll the source window until the program counter is displayed.

Toggles Menu

Show disassembly – Includes assembly code in the spurce window.
Start at 0x0000 – Begin program execution at location 0x0000.
Ignore asm conditionally assembled out – Reduces size of source window.

Int Menu

INT0-INT7 – force a hardware interrupt.
USB * - force a USB interrupt.

The V8JTAG Debugger

Debugging a v8 requires two microprocessors; a master and a slave. The master runs a special program which allows it to control the slave. A Win32 application (v8jtag.exe) communicates with the master via an RS-232 connection.

The v8jtag debugger allows the master to do the following to the slave:

- Start/stop/reset processor.
- Execute one clock, instruction, subroutine or c statement.
- Breakpoint on specified bus activity.
- Download a program.
- Examine/change program counter.
- Examine/change register values.
- Examine/change RAM.

Before debugging a program it must be compiled.

cv8 -g main.c

will create:

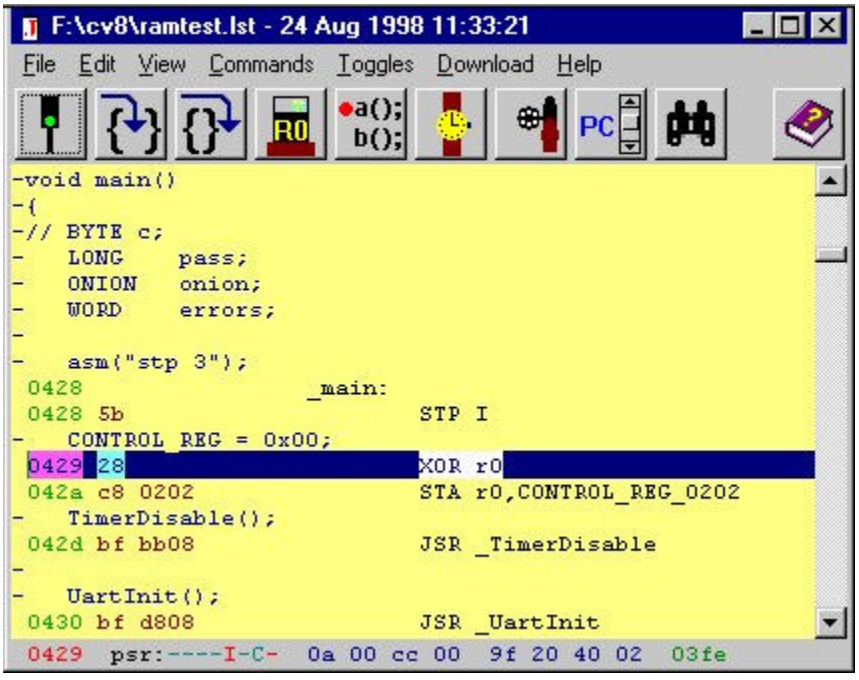
main.lst - unneeded
main.hex - an intel hex file
main.sdb - unneeded
main.sym - a list of symbols and their addresses

Then the resulting .hex and .sym files are combined with the .c source code files into a .lst file with the following command:

v8dis main.hex >main.lst

v8dis reads main.hex and main.sym as well as main.c and creates main.lst which is a combination of c and assembler code which the jtag debugger uses.

To debug a program, run v8jtag.exe. From the File menu select Open..., then navigate to and select main.lst.



Lines from c source files begin with a '-'. All other lines are V8 instructions.

The status bar at the bottom of the window indicates that:
0429 is the current Program Counter.
The I bit just became TRUE in the Processor Status Register (PSR).
The C bit is still TRUE in the PSR.
The Z bit just became FALSE in the PSR.
r0-r7 contain: 0a, 00, cc, 00, 9f, 20, 40, 02.
The Stack Pointer is 03fe.

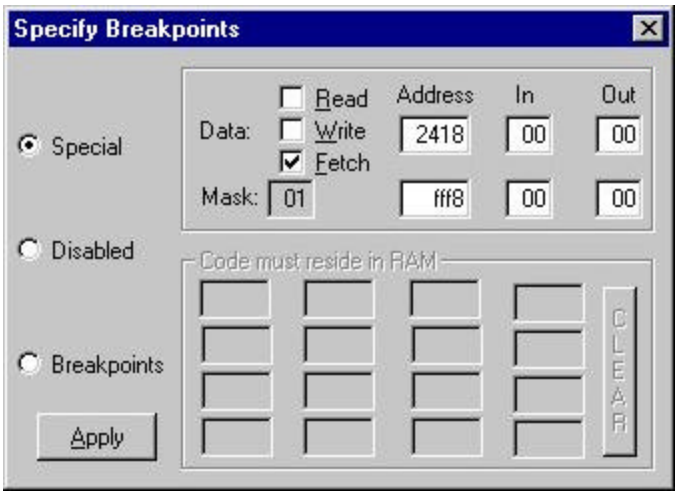
The buttons do the following (from left to right):
Start/Stop the processor.
Execute the next instruction/statement.
Execute the next instruction/statement but treat subroutines as one instruction.
Show/Hide the processor state window.
Show/Hide the breakpoint window.
Show/Hide the watchpoint window.
Reload the .hex and .lst files.
Scroll the source window until the program counter is displayed.
Find text in the source window.
Display help.

Breakpoints

V8jtag has one breakpoint command, but it is quite versatile. The internal breakpoint command consists of a seven byte description of the desired event and a seven byte mask. The following two structures show the layout of the seven bytes.

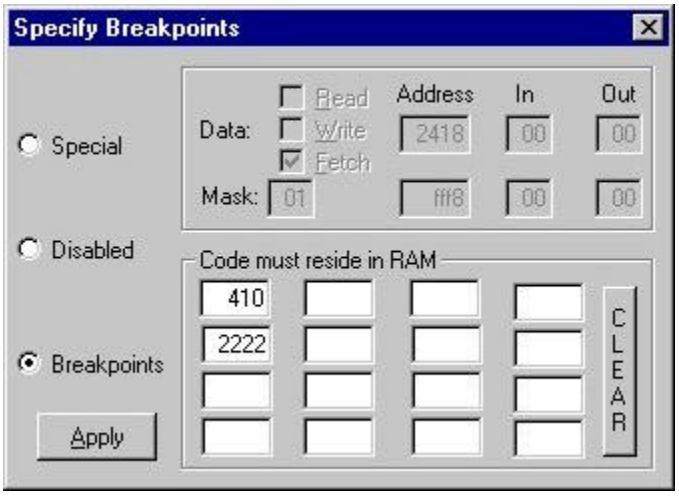
```
typedef union
        {
        BYTE    b;
        struct {char fetch:1,read:1,write:1,ready:1,reset:1;} bits;
        } STATUS;

typedef struct
        {
        BYTE    bInt;
        BYTE    bSetClrP;
        STATUS  status; // 4 = RESET, 3 = READY, 2 = WRITE, 1 = READ, 0 = FETCH
        BYTE    bDataOut;
        BYTE    bDataIn;
        WORD    address;
        } CTL, *PCTL;
```

The breakpoint window prompts for this data with the following screen:



The first two bytes of the CTL structure can be ignored. Byte three (status) lets you specify read, write, fetch or any memory access. Byte four (bDataOut) lets you break on a write of a specific value. Byte five (bDataIn) lets you break on a read of a specific value. Bytes six and seven (address) let you specify an address or range. In the above example we're looking for a fetch of any value from addresses 2418 – 241F.

Classic debuggers have allowed the user to halt execution at several different specific addresses. At first blush, this isn't possible – but we've got a solution. Suppose you want to halt execution at addresses 410 and 2222. Select the Breakpoints button in the following window and enter the desired breakpoints:



When you've specified breakpoints in this manner and then press the "Go" button v8jtag does the following:

Save the byte at each specified location.
Write 0xBB to each specified location.
Specify a breakpoint and mask combination which will run until a fetch of 0xBB.
Start the slave processor.
When a breakpoint is hit, the saved bytes are written back to their original locations.
The program counter is decremented by one.

We've added added a new opcode to the v8, BRK (value 0xBB) is a five clock no-op.

Due to the pipelining of the v8, when the slave realizes it has hit a breakpoint condition it takes two more clocks to actually halt the processor. If the breakpoint was on a simple instruction like "inc r0" the processor will halt on one of the subsequent two instructions. This is somewhat problematic if the instruction was a jmp/jsr/br0/br1. This is why the BRK instruction was added. Since BRK uses five clocks, subsequent instructions do not begin executing during the two clock window when the processor is halting. Since the program counter is incremented during the clock cycle where the fetch occurs, it is necessary to decrement it when we detect the breakpoint.

Menus

File Menu

Open – Specify the .LST file to use.
Reload – Reload and reparse the .LST and .HEX files.
Font – Specify the font to use.
Color – Specify source window colors.
Comm settings – Specify the comm port and its parameters.
Print comm window – Print the contents of the comm window.
Exit – Exit v8jtag.

Edit Menu

Find – Search source window for a specified string.
Find Next – Search source window for next occurance of string.
Edit Source – Open source code in your favorite editor.
The editor is specified in HKEY_LOCAL_MACHINE\Software\VAutomation Editor.
Clear Comm window – Remove all text from the comm window.

View Menu

Breakpoints – Show/Hide breakpoint window.
Comm port activity – Show/Hide comm port activity.
Processor – Show/Hide processor state window.
Watchpoints – Show/Hide watchpoint window.

Commands Menu

Clock 1 – Execute one clock cycle.
Step 1 – Execute one instruction/statement.
Step over JSR – Execute one instruction/statement but treat subroutines as one.
Go – Start slave processor.
Stop – Stop slave processor.
Scroll to PC – Scroll the source window until the program counter is displayed.
Abort download – Stop downloading.
Initialize master – Download jtag.hex to master.
Reset slave – Halt and reset slave processor.

Toggles Menu

Show disassembly – Show/Hide disassembly code in source window.
Start at 0x0000 – Begin program execution at location 0x0000.
Auto-write after download – After downloading send command to write EEPROM.

Download

Download .hex file to slave.

Help Menu

About v8jtag – shows file dates.
V8jtag help – displays help file.

HiTech Software CV8 C Compiler

The CV8 C compiler from HiTech software (HTTP://www.htsoft.com) is a full ANSII C compatible C compiler specifically designed for the V8-microRISC microprocessor. Details on the compiler and how to use it are available directly from HiTech software. The following paragraphs give a brief overview of how to use the CV8 compiler with VAutomations simulator and JTAG debugger.

UNDER CONSTRUCTION!!!